

Documentation API d'Interfaçage DGCC

■ Présentation

Cette API permet d'interfacer deux bases de données : une base distante (DGCC) et une base locale. Elle assure la récupération, la transformation et l'importation des documents, pièces et métadonnées.

■ Architecture

Base DGCC	<-->	API Node.js (Controllers / Services)	<-->	Base Locale
- documents		- dgcc.controller.js		- documents
- typeddocuments		- import.controller.js		- pieces
- metafields		- services / logger		- metafields
- pieces				- fichiers

■■ Configuration (.env)

```
# JWT
JWT_SECRET=*****
REFRESH_SECRET=*****

# Email
EMAIL_USER=*****
EMAIL_PASS=*****

# DB Locale
DB_HOST=mysql
DB_NAME=dgcc
DB_USER=root
DB_PASSWORD=root

# DB DGCC
DGCC_DB_HOST=localhost
DGCC_DB_NAME=document
DGCC_DB_USER=root
DGCC_DB_PASSWORD=root
```

■ Modèles de données

Principales tables DGCC : documents, typeddocuments, metafields, documentvalues, pieces, piece_meta_fields, piece_values, pieces_fichiers, document_fichiers.

La base locale reprend une structure similaire avec adaptation métier.

■ Endpoints API

GET /api/dgcc/full : Récupérer tous les documents complets

GET /api/dgcc/:id/full : Récupérer un document spécifique

POST /api/import/documents/all : Importer tous les documents

POST /api/import/documents/:documentId : Importer un document

■ Flux de données

1. Appel API DGCC
2. Mapping des données
3. Création type document
4. Création document
5. Import valeurs
6. Import fichiers (non bloquant)
7. Import pièces et relations

■ Logique d'import

La fonction centrale est `importSingleDocument()`. Elle gère la création des types, métadonnées, documents, valeurs et pièces.

Les fichiers sont téléchargés via Axios avec gestion d'erreur non bloquante.

■ Gestion des fichiers

```
downloadFile(url, destination, token)
- timeout: 10s
- retourne true/false
- non bloquant
```

■ Transactions

Import unitaire : transaction locale.

Import global : transaction globale avec rollback en cas d'erreur.

■■ Gestion des erreurs

Les erreurs critiques déclenchent un rollback.

Les erreurs de fichiers sont loguées mais non bloquantes.

■ Sécurité

Utilisation de JWT pour sécuriser les endpoints d'import.

Validation des données et nettoyage des chemins de fichiers.

Les variables sensibles doivent rester dans `.env`.

■ Logging

Logs structurés avec niveaux : succès, warning, erreur, debug.

■ Structure du projet

```
project/  
  ■■■ config/  
  ■■■ controllers/  
  ■■■ routes/  
  ■■■ models/  
  ■■■ dgcc_models/  
  ■■■ testImportation/  
  ■■■ .env
```

■ Conclusion

Cette API offre un mécanisme robuste d'interfaçage entre deux systèmes avec gestion des relations complexes, des fichiers et des transactions.